

Research Report: Toy Blockchain and Ledger Simulator

Golang Developer Assessment – Backend Engineering Internship

July 2026

Introduction

This report documents experiments run against my own implementation (`toyblockchain`), a single-process, proof-of-work blockchain and ledger written in Go, plus a short design write-up and a discussion of how this toy relates to production blockchains. Every number and every terminal transcript in this report was produced by actually running the code in this repository on my own machine, not estimated or reconstructed afterwards.

The implementation covers all required functional requirements (block structure, genesis block, deterministic hashing, transactions and a ledger, bounded proof-of-work mining, full-chain validation with tamper detection, a command-line interface, and JSON persistence), and also implements the digital-signatures stretch goal in full, plus two pieces of robustness that are not explicitly required but that felt necessary for a CLI tool that is expected to survive being interrupted mid-mine or run twice against the same file: atomic saves and a process-level file lock.

1. Tamper-evidence experiment

Setup. I built a small honest chain using the CLI (wallet creation, a faucet grant, a signed transfer, and two mined blocks), confirmed it validates, then edited the persisted chain file directly on disk with a text editor, simulating an attacker with raw file access.

Before tampering, the chain loads and validates cleanly:

```
PS D:\toyblockchain> go run . -data "data/chain.json" -wallets "data/wallets.json" validate
Loaded chain from data/chain.json (4 blocks).
Chain is VALID (4 blocks)
```

Tamper. I made a backup copy of the valid file, then opened `data/chain.json` in Notepad and hand-edited one transaction field inside an already-mined block, without touching that block's stored `hash` field and without re-mining anything.

After tampering, simply starting the program again (without even running the `validate` command) fails immediately:

```
PS D:\toyblockchain> go run . -data "data/chain.json" -wallets "data/wallets.json"
Error: loaded chain is invalid: block 2 invalid: stored hash does not match recomputed hash
exit status 1
```

Why this happens, precisely. `chain.Load` calls `Chain.Validate` before ever handing the loaded chain back to the caller, so a tampered file is rejected at load time, not only when a user happens to type `validate`. `Chain.Validate` recomputes each block's hash from its current on-disk fields (height, timestamp, transactions, previous hash, nonce, difficulty) and compares that recomputed hash against the `hash` field stored in the same block. Editing a transaction amount changes the JSON payload that feeds SHA-256, and by

the avalanche property of a cryptographic hash function this produces a completely different digest. Since I didn't also rewrite the stored `hash` field to match, the **hash-recomputation check** is what catches it, and it is caught at exactly the block I altered.

I restored the chain from the backup afterwards to keep working with a valid file.

A more careful attacker. The unit test suite goes one step further than my manual experiment and asks what happens if the attacker patches the block's stored hash to match the tampered content, so the hash-recomputation check alone would not catch anything (`TestTamperDetection` in `chain/chain_test.go` covers the amount-only case; both tests build a fresh honest chain per run and assert on the returned `ValidationError`):

```
testChain.Blocks[2].Transactions[0].Amount = 999999
err := testChain.Validate()
// -> &ValidationError{BlockHeight: 2, Reason: "stored hash does not match
recomputed hash"}
```

A second test, `TestTamperedDifficultyRejected`, goes further still: it lowers block 2's recorded `Difficulty` to 0 and recomputes the hash so it matches the new (lower-difficulty, and therefore much easier to produce) content:

```
testChain.Blocks[2].Difficulty = 0
testChain.Blocks[2].Hash = testChain.Blocks[2].ComputeHash()
err := testChain.Validate()
// -> &ValidationError{BlockHeight: 2, Reason: "recorded difficulty 0 does not
match expected difficulty 3"}
```

This still fails validation, but for a different reason. `Chain.Validate` does not trust a block's own recorded difficulty; for every block it independently recomputes the *expected* difficulty for that height from the chain's `DifficultySchedule` and compares the two before it even looks at the hash. So lowering the recorded difficulty to make re-mining cheap is caught immediately, and lowering it while leaving the hash alone would additionally fail the difficulty-target check (the hash would no longer satisfy the difficulty it claims). Either way, a real attacker is forced back to the same conclusion as the simple case above: the only way to make a tampered block internally consistent is to re-mine it at its true required difficulty, which then breaks the `prev_hash` link to the next block, which forces every block after it to be re-mined too. This is the same “re-mine the tampered block and everything after it” cost that makes real blockchains tamper-resistant, minus the added ingredient of a live network of peers who would simply reject the attacker's rewritten history in favour of the honest chain (see the discussion questions below for that distinction).

Both scenarios are also exercised as automated regression tests (`go test ./chain -run TestTamperDetection`, `go test ./chain -run TestTamperedDifficultyRejected`), so this behaviour is checked on every run, not only when I remember to try it by hand.

2. Difficulty versus effort experiment

Setup. I wrote a small standalone driver (kept outside the shipped CLI, in a throwaway `experiments/` directory, since it is not part of the deliverable) that mines fresh blocks directly through the `block` package's `Mine` method at difficulties 1 through 6, five trials per difficulty level, with a 30-second timeout per trial, and records the number of hash attempts and wall-clock time needed to find a valid nonce.

Results, averaged over the trials that completed within the timeout at each level:

Difficulty (leading hex zeros)	Avg. attempts	Avg. time	Trials completed
1	20	0.22 ms	5 / 5
2	42	~0 ms	5 / 5
3	1,635	2.17 ms	5 / 5
4	134,910	154.69 ms	5 / 5
5	63,976	73.43 ms	5 / 5
6	14,071,708	15.90 s	3 / 5

An honest note on difficulty 5. Read literally, the average for difficulty 5 (63,976 attempts) is *lower* than the average for difficulty 4 (134,910 attempts), which looks like it contradicts the trend. It doesn't, and I want to be explicit about why rather than quietly smoothing it over: the number of attempts needed to find a hash with a given number of leading zero hex digits follows a geometric distribution, which has high variance — five trials is a small sample, and difficulty 4's five trials happened to include one unusually expensive run (one trial alone took 177 ms against a mean of about 155 ms), while difficulty 5's five trials happened to land close together and on the low side. The expected value still grows with difficulty; a small sample of a high-variance process just doesn't always show a clean staircase. The difficulty 6 row makes the same point from the other direction: two of the five trials hit the 30-second timeout before finding a nonce at all, which is itself strong evidence of how much the *tail* of the distribution grows, even though the three that did complete average out to a number consistent with the overall trend.

I also have a handful of real single-sample data points from actually using the CLI, which corroborate the same shape without smoothing: mining two blocks at difficulty 3 during a normal session took 2,112 and 598 attempts respectively (2.85 ms and 1.12 ms), and a block mined shortly after at difficulty 4 took 70,104 attempts (89.04 ms) — all comfortably in the same order of magnitude as the table above, and all still small enough to finish in well under a second, which matters because the brief explicitly asks for difficulty low enough that the reviewer isn't left waiting minutes.

Trend. Despite the difficulty-5 sampling noise, the shape across all six rows is unmistakable: this is **not linear, it grows roughly exponentially**. Going from difficulty 1 to difficulty 6 multiplies the expected work by roughly 700,000x, not by 6x. That factor is exactly what theory predicts: each hex character of a SHA-256 digest is effectively uniformly random over 16 possible values (0-f), so requiring one more leading zero hex digit divides the probability that any single attempt succeeds by 16, and therefore multiplies the *expected* number of attempts needed by roughly 16. Concretely, requiring N leading hex zeros means a given attempt succeeds with probability $1/16^N$, so the expected number of attempts is on the order of 16^N — an exponential function of the difficulty, not a polynomial or linear one.

This is the whole point of proof-of-work as a rate-limiting mechanism: a small, linear increase in the *difficulty parameter* that the network agrees on translates into an exponential increase in the *computational cost* of finding a valid block, which is exactly what makes it expensive to rewrite history or to out-mine the honest network, and exactly why Section 1's "more careful attacker" scenario above is expensive rather than free.

3. Design write-up

Hashing scheme. Each block's hash is `SHA-256(JSON(payload))`, where `payload` is an explicitly-ordered struct (`hashPayload` in `block.go`) with six fields, in this exact order: `height`, `timestamp`, `transactions` (the full transaction list, each one serialised as `sender`, `recipient`, `amount`, `nonce`, `public_key`, `signature`, in that field order), `prev_hash`, `nonce`, `difficulty`. The

block's own `hash` field is deliberately excluded from this struct — it is the *output* of the computation, so including it would be circular. I chose `encoding/json` over a hand-rolled byte concatenation for three reasons: it's standard-library, Go's JSON encoder serialises struct fields in declaration order (never map order), which makes the output deterministic without any extra bookkeeping on my part, and it's trivial to eyeball or log the exact payload while debugging. The trade-off is that JSON is more verbose than a compact binary encoding, so hashing is marginally slower than it could be — a non-issue at this scale (see Section 2's timings), and not something I would optimise without a concrete reason to.

Signature scheme. Signatures are computed over a separate, narrower struct (`transactionPayload`: `sender`, `recipient`, `amount`, `nonce`) rather than over the whole `Transaction`, specifically so that the public key and signature fields are never inputs to the thing the signature is supposed to protect. I used Ed25519 (`crypto/ed25519`, standard library) over RSA or ECDSA because it has no parameters to choose incorrectly, has small fixed-size keys and signatures, and is fast enough that signing every CLI transaction is not observably slow. An address is `SHA-256(publicKey)[:20 bytes]`, hex-encoded — short enough to be readable in a terminal, long enough (160 bits) that collisions are not a practical concern for a toy of this size.

How validation guarantees integrity across the whole chain. `Chain.Validate` walks blocks from genesis to tip and, for every block, checks the following before moving on to the next block: (1) the block's recorded difficulty matches what the difficulty schedule says that height should require, (2) recomputing the hash from the block's current fields matches its stored hash, (3) that stored hash actually satisfies the required difficulty target, (4) either genesis-specific structural checks (height exactly 0, previous-hash equal to the fixed genesis value) or, for every later block, that height is exactly one more than the previous block's, that the timestamp is not earlier than the previous block's, and that `prev_hash` matches the previous block's *stored* hash exactly, and finally (5) every transaction in the block is replayed through a running `ledger.Ledger`, which itself checks signatures, sender ownership, transaction nonces (replay protection), and sender balances. Any single failure returns immediately with the offending block's height and a human-readable reason (`ValidationError`), which is how the acceptance criterion “identify the first block whose hash no longer matches” is satisfied even when the actual failure is not a hash mismatch at all (as Section 1's difficulty-tamper case shows).

The integrity guarantee comes from chaining checks (2) and the previous-hash check together: because each block's hash is computed over its own `prev_hash` field, and the next block stores that hash as its own `prev_hash`, every block is cryptographically bound to the one immediately before it. Changing anything inside block k changes block k 's hash (check 2 catches this immediately unless the attacker also re-mines), and even if an attacker patches block k 's stored hash to hide that, block $k+1$'s `prev_hash` now points to a hash that no longer exists on the patched chain — unless the attacker also re-mines block k to satisfy its required difficulty with the new content, and then updates and re-mines every block after it too, since each one's `prev_hash` needs to change to match. In other words, tampering with block k is only “free” if you are willing to redo the proof-of-work for every block from k to the tip, which is exactly the cost Section 2 measured.

4. Discussion questions

How does the previous-hash link make tampering with an old block impractical in a real chain, even though it is trivial in your local toy? Locally, “trivial” only in the sense that nothing stops me from editing the JSON file directly with a text editor, as Section 1 demonstrates. But even locally, a tamper that survives the hash-recomputation check still gets caught by the difficulty and proof-of-work checks, because faking a valid hash for new content requires re-mining, and the difficulty check independently verifies the recorded difficulty against the schedule regardless of whether the hash was patched. The real difference in a production chain is not that re-mining one block is somehow harder there than here — the difficulty of re-mining one block, for a given difficulty target, is the same computation regardless of who runs it. The difference is that re-mining one block isn't enough in a live network. Every peer independently holds a copy of the chain and only accepts the longest valid chain it sees. To rewrite an old block, an attacker has to re-mine that block *and every block after it* fast

enough to produce an alternative chain that overtakes the one the rest of the honest network is still extending in real time — a race against the combined hashing power of every other honest participant, not a one-off computation on a single laptop with no deadline. My toy has no peers and no “longest chain wins” rule, so nothing stops a locally-edited, internally-consistent chain from being accepted by validation once it is internally consistent again; a real network’s tamper-resistance comes from needing to out-mine everyone else simultaneously, not merely from the hash link by itself.

Proof-of-work is one way to decide who may add the next block. Name at least one alternative and give one advantage and one drawback versus proof-of-work. Proof-of-stake (used by, for example, Ethereum since its 2022 “Merge”) selects the next block proposer based on how much cryptocurrency they have locked up as stake, rather than how much computation they have burned searching for a nonce. *Advantage over proof-of-work:* it does not require racing enormous amounts of real-world energy to search for nonces, so it is far cheaper to run and does not scale its environmental footprint with network security the way proof-of-work does. *Drawback versus proof-of-work:* the cost of attacking the network is now denominated in the network’s own token rather than in an external, physical resource such as electricity and hardware, which raises subtler questions — for example “nothing at stake” style attacks, or the concern that wealth concentration in the token itself can concentrate block-producing power, whereas mining hardware and electricity are at least an independent cost that does not circularly depend on the value of the very chain being secured.

List three concrete ways your toy differs from a production blockchain. Pick one and sketch how you would add it.

1. *No consensus among peers.* There is exactly one process and one copy of the chain; there is no network protocol for peers to gossip blocks or agree on which chain is canonical, and therefore nothing resembling the “longest valid chain” discussion above.
2. *No Merkle tree.* A block’s transactions are hashed as a flat, directly embedded JSON list rather than summarised by a Merkle root, so there is no way for a light client to verify that one specific transaction is included in a block without downloading and hashing the entire block’s transaction list.
3. *No finality or fork-choice rule.* Because there is only ever one linear chain in memory, “validation passes” is treated as final. A real chain has to define what happens when two valid-looking chains compete for the same height, and when a block can be considered irreversible.

Sketch: adding a Merkle root. I would replace the flat `Transactions []Transaction` field inside `hashPayload` with a single `merkle_root` string field, computed by a new helper (for example `MerkleRoot(txs []Transaction) string`) that hashes each transaction individually, then repeatedly hashes adjacent pairs of hashes together (duplicating the last hash if the current level has an odd count) until exactly one hash remains — the standard pattern used by Bitcoin’s block header. `Block.ComputeHash` would then hash `height`, `timestamp`, `merkle_root`, `prev_hash`, `nonce`, and `difficulty`, and the full transaction list would still be stored on the block for local use, just no longer hashed directly. The payoff is that verifying “transaction X is in block Y” would only need `log2(n)` sibling hashes (a Merkle proof) rather than the whole block, which matters once blocks and chains are large enough that shipping the entire transaction list to every verifier is wasteful; for a local, single-machine toy at this scale, I judged the flat list to be the more readable and equally correct choice, so this stayed a sketch rather than an implementation.

5. Other behaviour worth recording

Two pieces of behaviour were not asked for directly in Section 7 of the brief, but came up while exercising the program and are relevant to the “honest about the limits of your solution” spirit of this report, so I am recording them here with real transcripts rather than leaving them purely as prose in the README.

Mining failure keeps pending transactions. Running with a deliberately tiny attempt budget (`-max-attempts 100 -max-nonce 100`) after raising the difficulty makes mining fail as expected, and the queued transaction is *not* silently lost:

```
> setdifficulty 8
Difficulty 8 will apply from block height 4
> faucet 3a47f5505930e7b01d869bb5664ad38adbe91983 1
Queued faucet grant: 3a47f5505930e7b01d869bb5664ad38adbe91983 receives 1
> mine
Error: maximum mining attempts reached
> pending
FAUCET -> 3a47f5505930e7b01d869bb5664ad38adbe91983 : 1 (nonce 0)
```

This matches `TestMiningFailureKeepsPendingTransactions` in `chain/chain_test.go`: a failed `MineBlock` call must not remove anything from `c.Pending` and must not append a half-mined block, so simply retrying with a larger budget (or a lower difficulty) picks up exactly where it left off instead of losing the queued transaction.

The file lock does what it says. Attempting to start a second instance of the CLI against a data file that is already locked by a running (or uncleanly terminated) process fails immediately and clearly, rather than silently loading stale state that a save from the first process would later overwrite:

```
PS D:\toyblockchain> go run . -lock-timeout 2s -data "data/chain.json" -wallets
"data/wallets.json"
Error: chain file is locked by another process
exit status 1
```

This is also covered by an automated test, `TestFileLockRejectsSecondProcess` in `chain/chain_test.go`, which starts two lock acquisitions against the same path in the same test process and asserts that the second one fails while the first is still held.

Sources

General blockchain concepts referenced above (proof-of-work as a difficulty-scaled search problem, the longest-valid-chain rule, and Merkle trees for compact transaction summaries) follow the design first described in Satoshi Nakamoto’s Bitcoin whitepaper, “Bitcoin: A Peer-to-Peer Electronic Cash System” (2008). The description of Ethereum’s move to proof-of-stake follows publicly documented information about “The Merge” from the Ethereum Foundation’s own documentation and blog posts. No text from either source is reproduced here; all explanations above are written in my own words based on general, publicly available knowledge of how these systems work.